Machine Learning (CSE_3148)

Mini Project Report

Smart Signal

A Walk-Forward Validated Machine Learning Swing-Trading System for NSE Equities

Group 17 · Section A

Project member	Registration No.
Arjit Menon	245805448
S. Darshan	245805023
Sreejith R. Pillai	245805025
Parthav U	245805188

GitHub repository: https://github.com/arjitmenon9/ml-trading-bot

Manipal Institute of Technology, Bengaluru

Date of submission: 30 October 2026

1. Abstract

This project builds an end-to-end machine learning system that decides when to buy National Stock Exchange (NSE) equities for short swing trades, executes those decisions through the Fyers broker API in paper-trading mode, and presents everything on a custom web dashboard called ATLAS. A first version framed the problem as three-class BUY/HOLD/SELL prediction on four stocks and reached only 37.3% test accuracy; in live use it bought two stocks and never sold them. The final version reframes the task as binary classification of trade outcomes: for every stock and every day, the label says whether buying at the next session's open and exiting with fixed take-profit, stop-loss and 15-day time rules would have made money after real brokerage charges. A bagged ensemble of three histogram gradient-boosted tree models is trained on 740,519 labelled stock-days covering 191 liquid NSE stocks from 2007 to October 2026, using 57 scale-free technical, market-regime and cross-sectional features. Evaluation uses a strict walk-forward protocol in which every test year is predicted by a model trained only on earlier data. On the 2022 to October 2026 holdout the system made 87 trades with an 87.4% win rate after charges, a profit factor of 2.04 and an average net return of 0.76% per trade; across all out-of-sample years from 2018 it made 133 trades with an 89.5% win rate. The report also discusses the main limitations: the model's overall ranking ability is weak (ROC AUC about 0.52), the edge is concentrated in the rare highest-confidence setups, trading is infrequent, and results are sensitive to position size because of fixed brokerage fees.

2. Introduction

Retail algorithmic trading in India has grown quickly because brokers such as Fyers expose order, quote and historical-data APIs to individual users. A common first approach is to combine technical indicators (moving averages, RSI, MACD and so on) into a hand-written score and trade when the score crosses a level. Such rules are easy to build, but their weights are guesses and they are rarely tested on data they were not designed on.

Machine learning offers a principled alternative: show a model many historical examples of indicator values together with what happened next, and let it learn the mapping. Financial time series, however, are noisy, non-stationary and heavily competed, so naive models easily look good in a random train/test split and then fail in real use because of look-ahead bias, overlapping labels or ignored trading costs.

This project started from an existing rule-based Fyers bot and a first ML model (version 1). Version 1 predicted BUY/HOLD/SELL over five days for four stocks (HDFCBANK, AUBANK, BAJAJFINSV, RELIANCE) using five years of data. It reached 37.3% accuracy (macro F1 0.359), barely above the 33% chance level for three classes. In live paper trading it bought HDFCBANK and AUBANK but never sold, because the model never output SELL for a held stock and the bot had no other exit. Its dashboard simulation used randomly generated prices, so its displayed win rate said nothing about the model. Version 2, described in this report, redesigns the learning problem, the data, the validation procedure, the trading loop and the user interface.

3. Objective

1. Reformulate the trading decision as a supervised learning problem whose label matches what the bot actually does (enter, then exit by explicit rules), including real Fyers brokerage and statutory charges.
2. Build a large, clean, split-adjusted historical dataset of NSE stocks and market indices from public sources.
3. Engineer scale-free features, including the indicators shown on the ATLAS dashboard (SMA 20/50/200, EMA 20), so that one model can learn across many stocks.
4. Train and select a model using strict walk-forward validation with an embargo, and choose the buy threshold without looking at the final test years.
5. Target an out-of-sample win rate of at least 85% after charges while keeping a positive profit factor.
6. Integrate the model into a live Fyers paper-trading bot with take-profit, stop-loss and time exits, persistent positions and robust handling of expired login tokens and API rate limits.
7. Integrate the ATLAS front end with a Python backend serving real market data, the model's view on every NSE stock, and three bot views: LIVE, SIM and LIVE SIM.

4. Data Source

Table 1. *Data sources.*

Source	Content	Use in the project
eod2 dataset (GitHub, BennyThadikaran/eod2_data)	NSE bhavcopy end-of-day OHLCV and delivery volume, adjusted for splits and bonuses; 191 stocks plus NIFTY 50, India VIX and 10 sector indices	Training and backtesting history up to 13 October 2026
Fyers History API	Daily candles for the same symbols and indices	Verification (matched eod2 to the paisa on 1,233 overlapping days per stock) and the daily top-up
Fyers Quotes API	Live last-traded price	Entries and exits in the live bot
Fyers charges list (fyers.in)	Brokerage, STT, exchange, SEBI, stamp duty, GST, DP fees	Cost model inside the label
Yahoo Finance (yfinance), Moneycontrol and Economic Times RSS, NSE	Global quotes, fundamentals, news, FII/DII flows	ATLAS website only (not used for training)

Preprocessing. Data before 2007 was dropped because the source contains unadjusted corporate actions in earlier years. Each stock's history was split into separate segments at any one-day move larger than 35% (demergers such as Adani Enterprises in 2015 or Tata Motors in 2025), so that no indicator or label crosses an artificial price jump; segments shorter than 300 days were discarded. Rows without a full year of history (needed for the 200-day average and its slope) were removed. The dataset used for the deployed model has 740,519 labelled stock-days for 191 stocks from 19 October 2007 to 13 October 2026. NIFTY 50 data starts in 2012 and India VIX in 2014; earlier rows keep those features as missing values, which the model handles natively.

Dataset summary. Each row is one stock on one trading day: 57 input features and one binary label (Section 5.1). The classes are imbalanced: 77.1% of all labelled stock-days are winning trades (positive class), yet buying every day loses 0.13% per trade on average because losses are larger than wins.

5. Methodology

5.1 Problem formulation and labelling

For a stock on day t, a hypothetical long trade is opened at the open of day t+1. Its take-profit is 0.75 × ATR% above entry and its stop-loss is 4 × ATR% below entry, where ATR% is the 14-day Average True Range as a percentage of price on day t (clamped to 0.8–6% and 2–15% respectively). The trade is simulated day by day on daily highs and lows: if a day touches both levels the stop is assumed to be hit first (pessimistic), gaps through a level fill at the open, and if neither level is hit within 15 days the trade exits at that day's close. The net return subtracts the full Fyers round-trip cost for a Rs. 50,000 delivery trade (0.446% including 0.05% slippage per side). The binary label is 1 if the net return is positive. This is a form of the triple-barrier labelling method of Lopez de Prado [1].

The asymmetric barriers raise the natural win rate (77.1% if one simply bought every day), but losses are larger than wins, so buying every day loses money. The model's job is to find the minority of setups where the win rate is high enough to make the expected value clearly positive.

5.2 Features

All 57 features are percentages, ratios or bounded oscillators so that stocks at different price levels are comparable:

- **39 stock features:** distances to EMA 9/20/21 and SMA 20/50/200, SMA 50 vs 200 gap and SMA 200 slope, RSI 2/5/14, MACD and histogram as % of price, Stochastic, Williams %R, CCI, ADX and DI difference, returns over 1/3/5/10/20/60 days, Bollinger %B and width, ATR%, volatility and its ratio, volume ratio, rolling VWAP gap, OBV change, internal bar strength, opening gap, down-streak, distance from 5-day low/high, 20-day z-score and distance from the 52-week high/low.
- **9 market features:** NIFTY 50 momentum, distance from its 200-day average, NIFTY RSI, India VIX level and change, and stock-versus-index relative strength.
- **9 cross-sectional features:** each stock's percentile rank among all stocks that day on seven measures, plus two market-breadth ratios.

5.3 Model selection

Version 1 compared logistic regression, random forest, extra trees and gradient boosting on 3,776 rows; histogram gradient boosting performed best. Version 2 keeps scikit-learn's HistGradientBoostingClassifier [4] because it trains quickly on 741,000 rows, captures non-linear interactions between indicators, and handles missing values without imputation. Three such models with different random seeds and 60% feature subsampling are averaged (bagging [5]), which reduced noise in the high-confidence tail and raised the out-of-sample profit factor

in our experiments. Hyperparameters: 300 iterations, learning rate 0.05, 31 leaves, at least 400 samples per leaf, L2 regularisation 1.0.

5.4 Walk-forward validation and threshold selection

For each test year Y from 2018 to 2026, a model is trained only on rows dated at least 30 days before 1 January of Y (an embargo longer than the 15-day label horizon, so no training label overlaps the test year) and then scores every stock-day of Y. Because the overall level of model scores drifts with the market regime, a fixed probability cut-off did not transfer between years (a cut-off chosen on 2018–2021 produced only 3 trades in 2022–2026). The bot therefore buys when a score is in the top 0.3% of all scores from the previous 250 trading days, a rolling and fully causal threshold. The quantile (0.997) was chosen using only the 2018–2021 out-of-sample trades, as the most active setting that reached a 92% win rate and a profit factor of at least 1.5 there. 2022 to October 2026 was then used as the holdout.

5.5 Trade gate and portfolio rules

Analysis of losing trades showed that they clustered in falling stocks and panicky markets. A rule-based gate therefore only allows trades when the stock's 200-day average is rising, price is no more than 10% below it, ATR is below 4% and India VIX is below 30. The model is trained on all rows (training only inside the gate performed worse) and the gate filters what it may buy. The portfolio rules mirror the live bot: one position per stock, at most 3 new trades per day (highest scores first) and at most 10 open positions.

5.6 Evaluation metrics

Because the bot acts only on a tiny fraction of predictions, plain classification accuracy is not informative. The project reports trade-level metrics: number of trades, win rate after charges, average net return per trade, profit factor (gross profits divided by gross losses) and total P&L at Rs. 50,000 per trade, together with the ROC AUC over all out-of-sample stock-days, win rate by confidence band, and a cost-sensitivity analysis.

6. Implementation Details

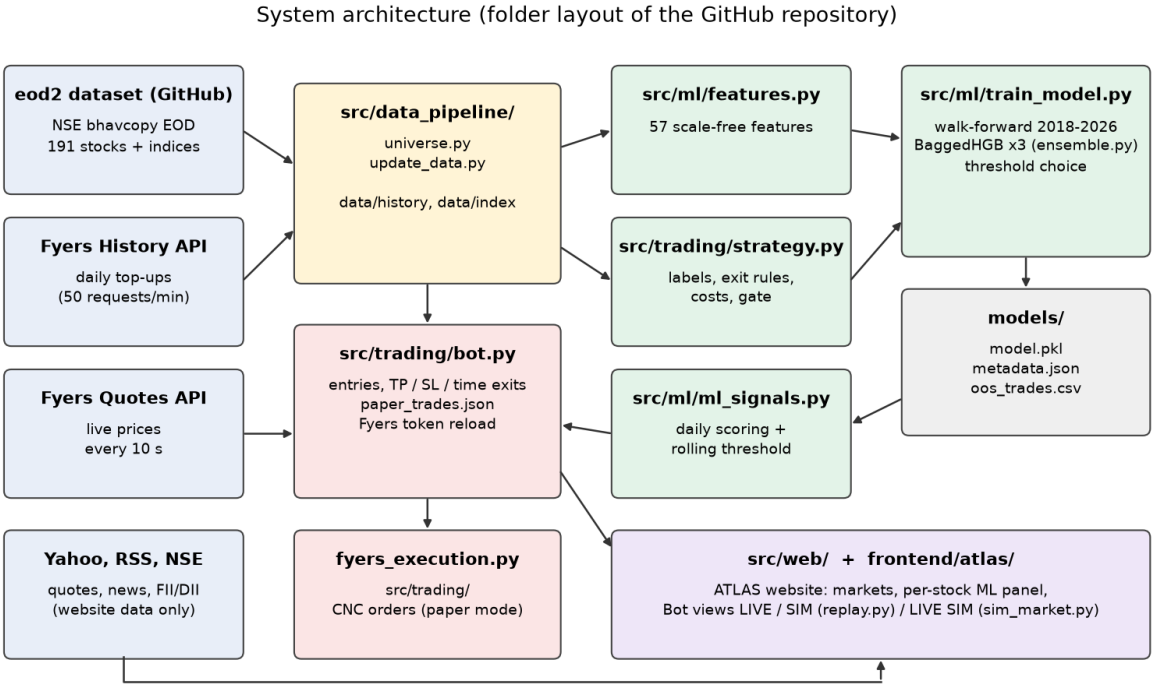


Figure 1. *System architecture, labelled with the folders of the GitHub repository.*

Table 2. *Implementation summary.*

Component	Implementation
Language and libraries	Python 3.12, pandas, NumPy, scikit-learn 1.9, joblib, Flask, fyers-apiv3, yfinance, matplotlib; HTML and JavaScript for the front end
Data layer (src/data_pipeline/)	universe.py (symbols, loading, segmenting); update_data.py (Fyers top-up paced under the 50 requests/minute limit, GitHub fallback)
Features and labels	src/ml/features.py (vectorised indicators, market and cross-sectional features); src/trading/strategy.py (exit rules, cost model, gate, trade simulator)
Training (src/ml/)	train_model.py (dataset build, walk-forward, threshold choice, final model); ensemble.py (BaggedHGB)
Evaluation	src/ml/backtest.py (yearly results, cost sensitivity, NIFTY benchmark); charts in reports/
Live inference	src/ml/ml_signals.py (scores the latest completed daily bar of every stock once a day, applies the gate, keeps the rolling threshold up to date)
Trading bot	src/trading/bot.py (morning data update and scoring, entries until 11:00, take-profit/stop-loss checks every 10 seconds, time stop at 15:20, positions saved to paper_trades.json, automatic reload of the daily Fyers token)
Execution	src/trading/fyers_execution.py (CNC delivery orders; LIVE_TRADING is off by default, so all orders are paper trades; kill-switch on daily loss and trade count)
Broker login	src/broker/ (config.py keeps credentials in a git-ignored fyers_secrets.json; auth.py and get_token.py perform the daily Fyers OAuth login)
Web application	src/web/atlas_api.py and market_data.py (Flask API with caching and failure back-off); frontend/atlas/ (ATLAS single-page site)

Component	Implementation
SIM and LIVE SIM engines	src/web/replay.py (replays the 133 real out-of-sample trades on real daily NSE prices); src/web/sim_market.py (a simulated 8-stock market traded by the bot 24/7)

6.1 Code repository

The complete source code, trained model, data and charts are on GitHub at https://github.com/arjitmenon9/ml-trading-bot. The repository is organised as follows: `src/` (Python code in five folders: trading, broker, ml, data_pipeline, web), `frontend/` (ATLAS website and the original dashboard), `scripts/` (one-click macOS launchers), `data/`, `models/`, `reports/`, `docs/` and `archive/` (the version 1 code, kept for comparison). Broker credentials, the trade journal and logs are git-ignored and never committed.

To reproduce the results:

1. Install the dependencies: `pip install -r requirements.txt`
2. Re-run the walk-forward training (20–40 minutes): `python src/ml/train_model.py`
3. Print the out-of-sample report and charts: `python src/ml/backtest.py`
4. List today's candidates: `python src/ml/predict_trend.py`
5. Run the website alone, without a broker login: `python src/web/atlas_server.py` (open http://127.0.0.1:5000)

6.2 Bot modes: LIVE, SIM and LIVE SIM

The ATLAS website (served by the bot at http://127.0.0.1:5000) shows global indices, Indian and US stocks, forex, commodities, crypto, India sector and breadth data, news and a stock screener. Every NSE stock page shows the model's estimated probability of a winning trade, whether the stock passes the gate and, for a qualifying setup, the planned entry, target and stop. The Trading bot page has three modes.

LIVE shows the running bot on the real market: Fyers quotes every 10 seconds, the day's model picks, open positions with their target and stop, the order log and today's P&L. Because the strategy buys only the top 0.3% of setups, on many days it correctly buys nothing.

SIM is the evidence view. replay.py plays the model's real walk-forward trades back in real time: each historical trading day lasts 45 seconds, prices tick along a path through that day's real open, high, low and close, BUY orders fire at the real entry price and SELL orders fire when the price reaches the trade's real exit level. Every number shown is a real out-of-sample result after charges.

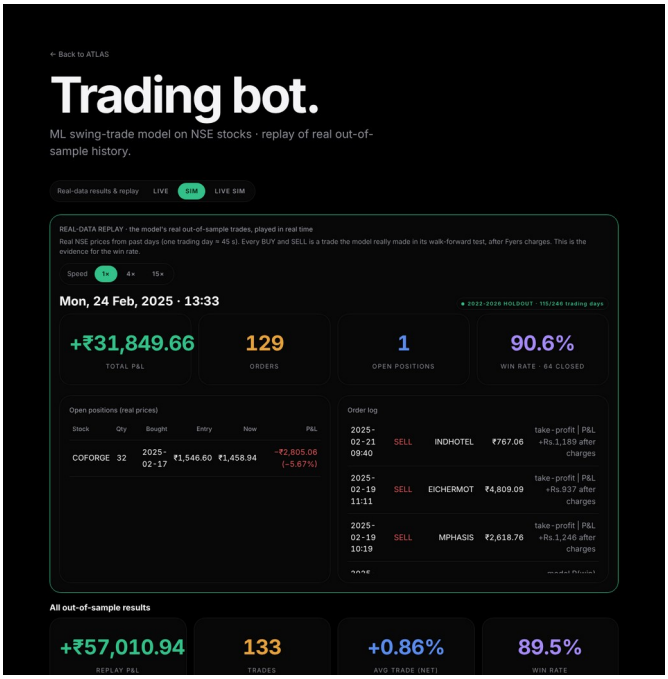


Figure 2. *SIM tab: the model's real out-of-sample trades replayed in real time on real NSE prices.*

LIVE SIM demonstrates the bot when the NSE is closed. sim_market.py generates a market for eight stocks in real time, one 5-minute candle per second, 24 hours a day; only each stock's starting price comes from its latest real close. Each simulated stock follows a slowly switching trend regime, mean reversion towards a drifting fair value, clustered volatility and rare news jumps. On every candle the bot computes indicators from the chart, applies the same trend and volatility gate as the real strategy, buys when a stock in an uptrend dips sharply (RSI 2 below 5 and price more than 2 standard deviations below its 20-bar mean), and sells at the strategy's take-profit, stop-loss or after 30 candles, paying the Fyers cost model.

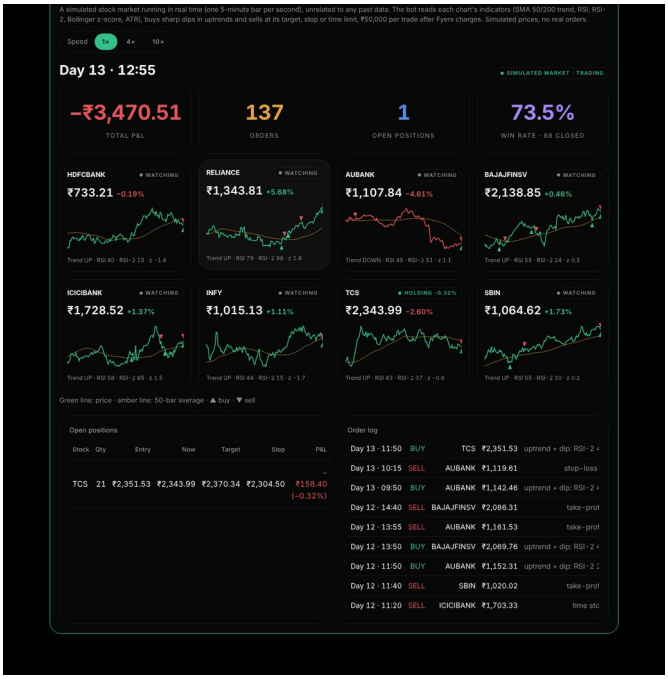


Figure 3. *LIVE SIM tab: the bot trading the simulated market, with charts, positions and order log.*

The LIVE SIM market's parameters were fixed in advance and not tuned to flatter the bot. Over five independent seeded runs of 20 simulated days each (487 closed trades) the bot won 81.5% of trades with an average net result of about −Rs. 27 per trade, i.e. roughly break-even. LIVE SIM therefore demonstrates that the bot reads charts and places and closes trades on its own in real time; it is not evidence of the model's accuracy, which comes only from the walk-forward results below.

7. Results and Discussion

7.1 Main results

Table 3. *Walk-forward results after Fyers charges.*

Period (all out-of-sample)	Trades	Win rate	Avg net / trade	Profit factor	P&L at Rs. 50,000 / trade
2018–2021 (used to choose the threshold)	46	93.5%	+1.04%	2.27	Rs. 24,012
2022 – Oct 2026 (holdout)	87	87.4%	+0.76%	2.04	Rs. 32,999
All 2018–2026	133	89.5%	+0.86%	2.12	Rs. 57,011
Version 1 model (for comparison)	n/a	37.3% accuracy (3-class)	n/a	n/a	n/a

Exits: 119 take-profit, 9 time stops, 5 stop-losses. The average holding period was 4.2 trading days. Winning trades averaged +1.8% and losing trades -7.2% net. The 95% confidence interval for the 89.5% win rate on 133 trades is roughly 83% to 94% (Wilson interval).

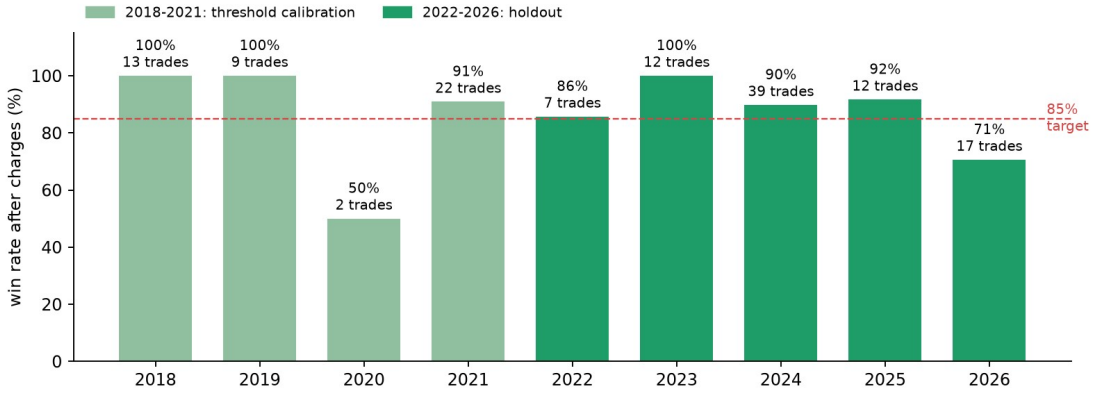


Figure 4. *Out-of-sample win rate per year (red line: 85% target).*

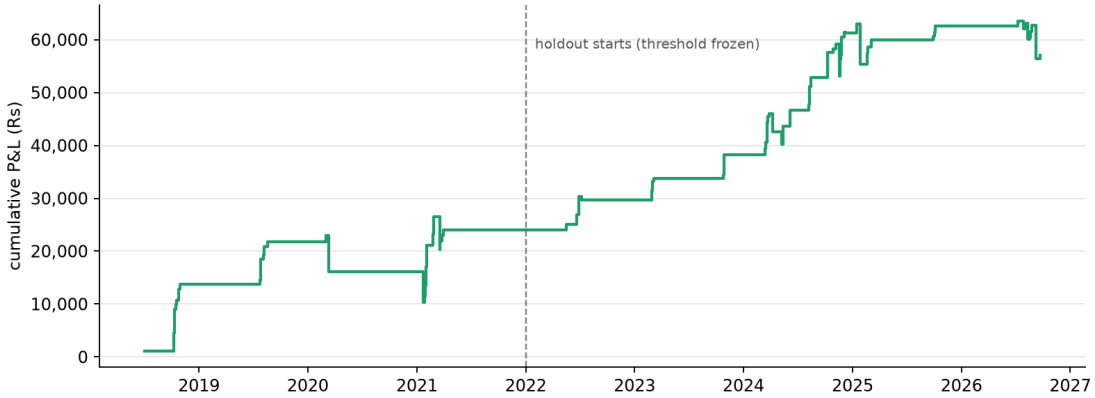


Figure 5. *Cumulative walk-forward P&L at Rs. 50,000 per trade after charges.*

7.2 Analysis of model quality

Across all 393,711 labelled out-of-sample stock-days the model's ROC AUC is only about 0.525 (between 0.48 and 0.57 per year), so its ability to rank ordinary days is weak, which is expected for daily stock returns. The useful signal lies in the extreme tail. Among stock-days in the top 0.3% of scores that also pass the gate, 85.3% won, against a 77.1% base rate; with the bot's rolling threshold the 750 gated candidates won 87.3%, and the portfolio rules (best 3 per day, one position per stock, at most 10 open) raise this to 89.5% on the trades actually taken. Figure 6 shows that high-confidence setups outside the gate do not achieve this, which is why both the model and the gate are needed. As a reproducibility check, the full walk-forward was re-run from scratch on data up to 22 October 2026 and selected the same quantile (0.997) and the same 133 trades with the same 89.5% win rate.

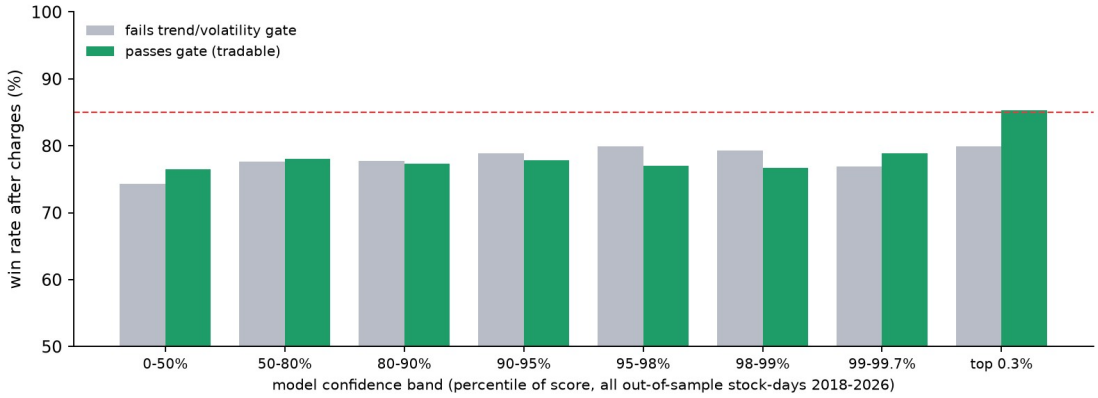


Figure 6. *Out-of-sample win rate by model confidence band, inside and outside the gate.*

7.3 Effect of trading costs

Table 4. *The same 133 trades at different position sizes.*

Rs. per trade	Round-trip charges	Win rate	Avg net / trade	Profit factor
5,000	1.33%	85.0%	−0.02%	0.97
10,000	0.94%	88.7%	+0.36%	1.44
25,000	0.57%	89.5%	+0.73%	1.95
50,000	0.45%	89.5%	+0.86%	2.12
1,00,000	0.38%	89.5%	+0.92%	2.22

Fyers charges a flat Rs. 20 per order and Rs. 14.75 per sell for depository fees, so small trades lose most of the edge: at Rs. 5,000 per trade the same signals are slightly loss-making. Over the 8.2 years tested, the strategy earned Rs. 57,011 while needing up to Rs. 5 lakh reserved (about 1.3% per year, simple), whereas NIFTY 50 buy-and-hold returned about 10% per year over the same window.

7.4 Novelties

- Labels that match the executed strategy, including the exact Fyers fee schedule, instead of generic up/down labels.
- A rolling, causal score-quantile threshold that adapts to regime drift, chosen only on calibration years.
- Cross-sectional rank and breadth features computed across the whole universe each day, both in training and live.
- A complete working system: live Fyers paper trading with token hot-reload, rate-limit pacing and persistent positions.
- The ATLAS dashboard with a per-stock model panel, a SIM mode that replays the model's real out-of-sample trades in real time, and a 24/7 LIVE SIM mode for demonstrations.

7.5 Limitations

- Not every year reaches 85%: 2020 (2 trades, 50%) and 2026 so far (17 trades, 70.6%) fall short.
- Weak overall discrimination (AUC about 0.52); the edge depends on a small number of very selective trades.
- Infrequent trading (about 16 trades a year) and modest total profit compared with simply holding the index.
- Survivorship bias, because the universe is today's large companies.
- The design (exit rules, gate, bagging, threshold target) was explored on the same history, so the holdout is strong but not perfectly untouched evidence.
- Fills at the open and at exact target/stop levels are assumed; slippage beyond 0.05% per side is not modelled.
- LIVE SIM uses a simplified rule on synthetic intraday candles rather than the daily ML model, so it is a demonstration only.

8. Conclusion and Future Scope

Reformulating the problem from three-class direction prediction to cost-aware trade-outcome classification, enlarging the dataset about 200-fold and validating strictly walk-forward turned a model with 37% accuracy and no working exits into a system that won 87.4% of its trades after charges on the 2022–2026 holdout, with a profit factor above 2. The project also delivers a working paper-trading bot and an integrated dashboard. The main lesson is that a high win rate is achievable only through selectivity, and must always be read together with profit factor, trade frequency and costs.

Future scope: intraday entry timing to reduce gap risk; survivorship-free index membership history; probability calibration and conformal methods for the threshold; position sizing by confidence and volatility; short-selling and sector-neutral variants; regime detection to pause trading in years like 2020; and several months of forward paper trading before any decision to use real money.

9. References

[1] M. Lopez de Prado, Advances in Financial Machine Learning. Hoboken, NJ: Wiley, 2018 (triple-barrier labelling, purging and embargo).

[2] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," Annals of Statistics, vol. 29, no. 5, pp. 1189–1232, 2001.

[3] G. Ke et al., "LightGBM: A highly efficient gradient boosting decision tree," in Advances in Neural Information Processing Systems 30, 2017.

[4] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[5] L. Breiman, "Bagging predictors," Machine Learning, vol. 24, no. 2, pp. 123–140, 1996.

[6] J. W. Wilder, New Concepts in Technical Trading Systems. Greensboro, NC: Trend Research, 1978 (RSI, ATR, ADX).

[7] J. Bollinger, Bollinger on Bollinger Bands. New York: McGraw-Hill, 2001.

[8] L. Connors and C. Alvarez, Short Term Trading Strategies That Work. TradingMarkets, 2009 (RSI-2 mean reversion).

[9] B. Thadikaran, "eod2_data: NSE end-of-day data," GitHub repository, https://github.com/BennyThadikaran/eod2_data (accessed 23 Oct 2026).

[10] FYERS, "Charges list," https://fyers.in/charges-list (accessed 23 Oct 2026).

[11] FYERS, "API v3 documentation," https://myapi.fyers.in/docsv3 (accessed Oct 2026).

[12] R. Aroussi, "yfinance," Python package, https://github.com/ranaroussi/yfinance.

[13] Project source code: Group 17, "Smart Signal / ATLAS ML trading bot," GitHub repository, https://github.com/arjitmenon9/ml-trading-bot.